



برنامه نویسی به زبان سی

کاراکترها و رشته ها

دکتر مهران صفایانی

safayani@iut.ac.ir

safayani.iut.ac.ir



<https://www.aparat.com/mehran.safayani>



https://github.com/safayani/Programming_Basics_course



Fundamentals of Characters and Strings

- **Characters:**

- The fundamental building blocks of source code.
- A **character constant** (e.g., 'z', '\n') is an int value represented in single quotes.

- **Strings:**

- A series of characters treated as a single unit.
- A **string literal** is written in double quotation marks (e.g., "John Q. Doe").

-

Critical String Concept: The Null Terminator

- **Rule:** Every string **must** end with the **null character** ('\0').
- **Danger:** Printing a non-null-terminated "string" is a logic error with undefined behavior:
 - It may print garbage data until a null byte is found.
 - It may cause a program crash ("segmentation fault" or "access violation").

Strings, Arrays, and Pointers

- A string is accessed via a pointer to its first character.
- A string's "value" is the address of its first character.
- Strings are essentially arrays of characters.

// Creates a modifiable 5-element array: 'b','l','u','e','\0'

`char color[] = "blue";`

// Creates a pointer to the first char of an IMMUTABLE string

`const char *colorPtr = "blue";`

Important Warnings and Best Practices

- **1. String Literals Are Immutable:**

- The C standard states that string literals (e.g., "blue") are **not modifiable**.
- To modify a string, you **must** store it in a character array.

```
char *str = "blue";  
str[0] = 'h'; // This causes UNDEFINED BEHAVIOR (usually crashes)
```

```
char str[] = "Hello World";
```

```
// or
```

```
char str[12] = "Hello World";
```

```
str[0] = 'h'; // This works fine - changes to "hello World"
```

- **2. Always Reserve Space for the Null Terminator:**

- The character array must be large enough to hold the string **and** the '\0'.
- A string longer than the array will overwrite other memory (a serious error).

Reading Strings Safely with scanf

- Use scanf to read a string into a character array.
- The & is not needed because the array name is a pointer.

- **Crucial Safety Measure: Use a Field Width!**

```
char word[20];
```

```
scanf("%19s", word); // Reads at most 19 characters
```

- **Why?** scanf reads until whitespace or EOF. The field width (%19s) prevents reading more characters than the array can hold, reserving one spot for the null terminator.
- **Without it**, excess input can overwrite memory and crash your program.

Practice: scanf() Behavior Study

```
#include "stdio.h"
void main(){
    char str1[]="hello";
    char str2[]="hi";
    printf("Before input\n str1=%s\n str2=%s\n",str1,str2);
    printf("Input for str2=");
    scanf("%s",str2);
    printf("After input\n str1=%s\n str2=%s\n",str1,str2);
}
```

To do

- Run the program multiple times, providing the inputs **12**, **123**, **1234**, and **12345** respectively, and analyze the output.
- In the scanf function, use the **scanf("%2s",str2)**, and then repeat previous step.

```
Before input
str1=hello
str2=hi
Input for str2=12
After input
str1=hello
str2=12
```

```
Before input
str1=hello
str2=hi
Input for str2=123
After input
str1=
str2=123
```

```
Before input
str1=hello
str2=hi
Input for str2=1234
After input
str1=4
str2=1234
```

```
Before input
str1=hello
str2=hi
Input for str2=12345
After input
str1=45
str2=12345
```

`scanf("%2s",str2);`

```
Before input
str1=hello
str2=hi
Input for str2=12
After input
str1=hello
str2=12
```

```
Before input
str1=hello
str2=hi
Input for str2=123
After input
str1=hello
str2=12
```

```
Before input
str1=hello
str2=hi
Input for str2=1234
After input
str1=hello
str2=12
```

```
Before input
str1=hello
str2=hi
Input for str2=12345
After input
str1=hello
str2=12
```


Character-Handling Library (<ctype.h>) (Part 1)

Function	Purpose	Checks For
isdigit	Is it a digit?	0 - 9
isalpha	Is it a letter?	A-Z, a-z
isalnum	Is it alphanumeric?	A-Z, a-z, 0-9
isxdigit	Is it a hexadecimal digit?	A-F, a-f, 0-9

`isdigit('8')` // Returns true (non-zero)

`isalpha('8')` // Returns false (0)

Character-Handling Library (<ctype.h>)(Part 2)

• Classification Functions:	Function	Purpose	Checks For
	islower	Is it a lowercase letter?	a - z
	isupper	Is it an uppercase letter?	A - Z

• Conversion Functions:

Function	Purpose
tolower	Converts an uppercase letter to lowercase. Returns the character unchanged if not uppercase.
toupper	Converts a lowercase letter to uppercase. Returns the character unchanged if not lowercase.

Character-Handling Library (<ctype.h>)(Part 3)

Function	Purpose	Checks For
isspace	Is it a whitespace character?	space, \f, \n, \r, \t, \v
iscntrl	Is it a control character?	\t, \v, \f, \a, \b, \r, \n
ispunct	Is it a punctuation character?	Printing character that is not a space, digit, or letter (e.g., \$, #, :)
isprint	Is it a printable character?	Any character that can be displayed, including a space.
isgraph	Is it a printable character?	Any character that can be displayed, excluding a space.

```

1 // fig08_01.c
2 // Using functions isdigit, isalpha, isalnum, and isxdigit
3 #include <ctype.h>
4 #include <stdio.h>
5
6 int main(void) {
7     printf("%s\n%s%s\n%s%s\n\n", "According to isdigit: ",
8         isdigit('8') ? "8 is a " : "8 is not a ", "digit",
9         isdigit('#') ? "# is a " : "# is not a ", "digit");
10
11     printf("%s\n%s%s\n%s%s\n%s%s\n\n", "According to isalpha:",
12         isalpha('A') ? "A is a " : "A is not a ", "letter",
13         isalpha('b') ? "b is a " : "b is not a ", "letter",
14         isalpha('&') ? "& is a " : "& is not a ", "letter",
15         isalpha('4') ? "4 is a " : "4 is not a ", "letter");
16
17     printf("%s\n%s%s\n%s%s\n%s%s\n\n", "According to isalnum:",
18         isalnum('A') ? "A is a " : "A is not a ", "digit or a letter",
19         isalnum('8') ? "8 is a " : "8 is not a ", "digit or a letter",
20         isalnum('#') ? "# is a " : "# is not a ", "digit or a letter");
21
22     printf("%s\n%s%s\n%s%s\n%s%s\n\n", "According to isxdigit:",
23         isxdigit('F') ? "F is a " : "F is not a ", "hexadecimal digit",
24         isxdigit('J') ? "J is a " : "J is not a ", "hexadecimal digit",
25         isxdigit('7') ? "7 is a " : "7 is not a ", "hexadecimal digit",
26         isxdigit('$') ? "$ is a " : "$ is not a ", "hexadecimal digit",
27         isxdigit('f') ? "f is a " : "f is not a ", "hexadecimal digit");
28 }

```

According to isdigit:

8 is a digit

is not a digit

According to isalpha:

A is a letter

b is a letter

& is not a letter

4 is not a letter

According to isalnum:

A is a digit or a letter

8 is a digit or a letter

is not a digit or a letter

According to isxdigit:

F is a hexadecimal digit

J is not a hexadecimal digit

7 is a hexadecimal digit

\$ is not a hexadecimal digit

f is a hexadecimal digit

String-Conversion Functions (<stdlib.h>) - Overview

- **Purpose:** Convert strings of digits into integer and floating-point values.
- **Library:** General Utilities Library (<stdlib.h>)
- **Key Functions:**
 - strtod -> Converts a string to a double
 - strtol -> Converts a string to a long int
 - strtoul -> Converts a string to an unsigned long int
 - strtoll / strtoull -> For long long and unsigned long long (C99+)

Function: strtod (String to Double)

- **Prototype:** `double strtod(const char *str, char **endPtr);`
- **Purpose:** Converts a sequence of characters representing a floating-point number to a double.
- **Arguments:**
 - str: The string to convert.
 - endPtr: A pointer to a char*. The function sets this pointer to the location of the **first character after** the converted number in the string.
- **Behavior:**
 - Ignores leading whitespace.
 - Returns 0.0 if no conversion is possible.
 - The endPtr is useful for parsing the rest of the string after the number.

```
char string[] = "123.45abc";
```

```
char *remainder;
```

```
double d = strtod(string, &remainder); // d = 123.45 remainder points to "abc"
```

```

1 // fig08_04.c
2 // Using function strtod
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(void) {
7     const char *string = "51.2% are admitted";
8     char *stringPtr = NULL;
9
10    double d = strtod(string, &stringPtr);
11
12    printf("The string \"%s\" is converted to the\n", string);
13    printf("double value %.2f and the string \"%s\" \n", d, stringPtr);
14 }

```

The string "51.2% are admitted" is converted to the double value 51.20 and the string "% are admitted"

Function: strtol (String to Long Integer)

- **Prototype:** `long int strtol(const char *str, char **endPtr, int base);`
- **Purpose:** Converts a sequence of characters representing an integer to a long int.
- **Arguments:**
 - str: The string to convert.
 - endPtr: Same as in strtod (can be NULL to ignore).
 - base: The number base (radix) of the value to convert (e.g., 10 for decimal, 16 for hexadecimal).
- **Key Feature:**
 - A base of **0** allows for automatic detection:
 - Leading 0x or 0X -> Hexadecimal (base 16)
 - Leading 0 -> Octal (base 8)
 - Otherwise -> Decimal (base 10)
 - Supports bases from 2 to 36. Letters A-Z (or a-z) represent values 10 to 35.

```
long x = strtol("1A3F", NULL, 16); // x = 6719 (decimal)
```



```

1 // fig08_05.c
2 // Using function strtol
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(void) {
7     const char *string = "-1234567abc";
8     char *remainderPtr = NULL;
9
10    long x = strtol(string, &remainderPtr, 0);
11
12    printf("%s\ "%s\ "\n%s%ld\n%s\ "%s\ "\n%s%ld\n",
13        "The original string is ", string,
14        "The converted value is ", x,
15        "The remainder of the original string is ", remainderPtr,
16        "The converted value plus 567 is ", x + 567);
17 }

```

```

The original string is "-1234567abc"
The converted value is -1234567
The remainder of the original string is "abc"
The converted value plus 567 is -1234000

```

Input Function: fgets

- **Prototype:** `char *fgets(char *str, int n, FILE *stream);`
- **Purpose:** Safely reads a line of text from a stream (e.g., stdin) into a string.
- **Key Features:**
 - Reads until **newline**, **end-of-file**, or until `n-1` characters are read.
 - **Appends a null terminator** (`\0`) automatically.
 - Includes the newline character in the string if it is read.
 - The `n` argument prevents buffer overflow.

```
char line[100];
```

```
fgets(line, 100, stdin); // Reads a line from the keyboard
```

Output Functions: putchar and puts

- putchar
 - **Prototype:** `int putchar(int c);`
 - **Purpose:** Writes a single character to the standard output.
 - **Example:** `putchar('A');` // Prints A
- puts
 - **Prototype:** `int puts(const char *str);`
 - **Purpose:** Writes a string to the standard output.
 - **Key Feature:** Automatically appends a **newline character** after printing the string.
 - **Example:** `puts("Hello");` // Prints "Hello" followed by a newline

Case Study: Recursive String Reversal

- **Demonstrates:** Using `fgets` and `putchar` in a recursive algorithm.
- **Process:**
 - `fgets` reads a line from `stdin`.
 - Recursive function `reverse` calls itself until it finds the null terminator.
 - On the way back from recursion, `putchar` prints each character, resulting in reversed output.
- **Note:** This is a demonstration of recursion; in practice, an iterative loop is more efficient for reversing a string.

```

1 // fig08_07.c
2 // Using functions fgets and putchar
3 #include <stdio.h>
4 #define SIZE 80
5
6 void reverse(const char * const sPtr);
7
8 int main(void) {
9     char sentence[SIZE] = "";
10
11     puts("Enter a line of text:");
12     fgets(sentence, SIZE, stdin); // read a line of text
13
14     printf("\n%s", "The line printed backward is:");
15     reverse(sentence);
16     puts("");
17 }
18
19 // recursively outputs characters in string in reverse order
20 void reverse(const char * const sPtr) {
21     // if end of the string
22     if ('\0' == sPtr[0]) { // base case
23         return;
24     }
25     else { // if not end of the string
26         reverse(&sPtr[1]); // recursion step
27         putchar(sPtr[0]); // use putchar to display character
28     }
29 }

```

Enter a line of text:
Characters and Strings

The line printed backward is:
sgnirtS dna sretcarahC

Input Function: getchar

- **Prototype:** `int getchar(void);`
- **Purpose:** Reads the next character from the standard input.
- **Key Features:**
 - Returns the character as an int to support the EOF indicator.
 - Often used in loops to read character-by-character input.

```
char sentence[80];  
int i = 0, c;  
while ((c = getchar()) != '\n' && i < 79) {  
    sentence[i++] = c;  
}  
sentence[i] = '\0'; // Null-terminate the string
```

```

1 // fig08_08.c
2 // Using function getchar
3 #include <stdio.h>
4 #define SIZE 80
5
6 int main(void) {
7     int c = 0; // variable to hold character input by user
8     char sentence[SIZE] = "";
9     int i = 0;
10
11     puts("Enter a line of text:");
12
13     // use getchar to read each character
14     while ((i < SIZE - 1) && (c = getchar()) != '\n') {
15         sentence[i++] = c;
16     }
17
18     sentence[i] = '\0'; // terminate string
19
20     puts("\nThe line entered was:");
21     puts(sentence); // display sentence
22 }

```

Enter a line of text:
This is a test.

The line entered was:
This is a test.

In-Memory Formatting: sprintf

- **Prototype:** `int sprintf(char *str, const char *format, ...);`
- **Purpose:** Writes **formatted output into a string** (like printf but to a buffer).
- **Use Case:** Composing strings with numbers and other variables.
- **Critical:** Ensure the target array is large enough to hold the resulting string.

```
char s[100];
```

```
int x = 10;
```

```
double y = 3.5;
```

```
sprintf(s, "x = %d, y = %.2f", x, y); // s now contains: "x = 10, y = 3.50"
```



```

1 // fig08_09.c
2 // Using function sprintf
3 #include <stdio.h>
4 #define SIZE 80
5
6 int main(void) {
7     int x = 0;
8     double y = 0.0;
9
10    puts("Enter an integer and a double:");
11    scanf("%d%lf", &x, &y);
12
13    char s[SIZE] = {'\0'}; // create char array
14    sprintf(s, "integer:%6d\ndouble:%7.2f", x, y);
15
16    printf("The formatted output stored in array s is:\n%s\n", s);
17 }

```

```

Enter an integer and a double:
298 87.375
The formatted output stored in array s is:
integer:    298
double:   87.38

```

In-Memory Scanning: sscanf

- **Prototype:** `int sscanf(const char *str, const char *format, ...);`
- **Purpose:** Reads **formatted input from a string** (like `scanf` but from a buffer).
- **Use Case:** Parsing data from a string into variables.

```
char s[] = "123 45.6";
```

```
int x;
```

```
double y;
```

```
sscanf(s, "%d %lf", &x, &y); // x = 123, y = 45.6
```

```

1 // fig08_10.c
2 // Using function sscanf
3 #include <stdio.h>
4
5 int main(void) {
6     char s[] = "31298 87.375";
7     int x = 0;
8     double y = 0;
9
10    sscanf(s, "%d%lf", &x, &y);
11    puts("The values stored in character array s are:");
12    printf("integer:%6d\ndouble:%8.3f\n", x, y);
13 }

```

```

The values stored in character array s are:
integer: 31298
double:  87.375

```

String-Manipulation Library (<string.h>) - Overview

- **Purpose:** Provides functions for copying, concatenating, and manipulating string data.
- **Library:** String-Handling Library (<string.h>)
- **Key Functions Covered:**
 - Copying: strcpy, strncpy
 - Concatenation: strcat, strncat

String Copying Functions

- **strcpy**
 - **Prototype:** `char *strcpy(char *dest, const char *src);`
 - **Purpose:** Copies the entire source string (including the null terminator) into the destination.
 - **Critical Warning:** The programmer **must** ensure the destination array is large enough to store the entire source string. Otherwise, a buffer overflow occurs.
- **strncpy**
 - **Prototype:** `char *strncpy(char *dest, const char *src, size_t n);`
 - **Purpose:** Copies up to n characters from the source to the destination.
 - **Key Behavior:**
 - **Does NOT automatically append a null terminator** if n is less than or equal to the source string's length.
 - If the source string is shorter than n, the remainder of the destination is padded with null characters.
- **Best Practice:** Always manually null-terminate the destination if there's any doubt.

```
char z[15];
```

```
strncpy(z, x, 14);
```

```
z[14] = '\0'; // Manually null-terminate
```



```

1 // fig08_11.c
2 // Using functions strcpy and strncpy
3 #include <stdio.h>
4 #include <string.h>
5 #define SIZE1 25
6 #define SIZE2 15
7
8 int main(void) {
9     char x[] = "Happy Birthday to You"; // initialize char array x
10    char y[SIZE1] = ""; // create char array y
11    char z[SIZE2] = ""; // create char array z
12
13    // copy contents of x into y
14    printf("%s%s\n%s%s\n",
15        "The string in array x is: ", x,
16        "The string in array y is: ", strcpy(y, x));
17
18    strncpy(z, x, SIZE2 - 1); // copy first 14 characters of x into z
19    z[SIZE2 - 1] = '\0'; // terminate string in z, because '\0' not copied
20    printf("The string in array z is: %s\n", z);
21 }

```

```

The string in array x is: Happy Birthday to You
The string in array y is: Happy Birthday to You
The string in array z is: Happy Birthday

```

String Concatenation Functions

- `strcat`

- **Prototype:** `char *strcat(char *dest, const char *src);`
- **Purpose:** Appends the entire source string to the end of the destination string.
- **Process:** Overwrites the destination's null terminator and adds a new one after the appended string.
- **Critical Warning:** The programmer **must** ensure the destination array is large enough to hold the combined result of both strings.

- `strncat`

- **Prototype:** `char *strncat(char *dest, const char *src, size_t n);`
- **Purpose:** Appends up to `n` characters from the source string to the destination.
- **Key Behavior:** **Always appends a null terminator** after the concatenated characters, making it safer than `strncpy`.
- **Best Practice:** Preferred over `strcat` when controlling the number of characters to append is important for safety.

```

1 // fig08_12.c
2 // Using functions strcat and strncat
3 #include <stdio.h>
4 #include <string.h>
5
6 int main(void) {
7     char s1[20] = "Happy "; // initialize char array s1
8     char s2[] = "New Year "; // initialize char array s2
9     char s3[40] = ""; // initialize char array s3 to empty
10
11     printf("s1 = %s\ns2 = %s\n", s1, s2);
12
13     // concatenate s2 to s1
14     printf("strcat(s1, s2) = %s\n", strcat(s1, s2));
15
16     // concatenate first 6 characters of s1 to s3
17     printf("strncat(s3, s1, 6) = %s\n", strncat(s3, s1, 6));
18
19     // concatenate s1 to s3
20     printf("strcat(s3, s1) = %s\n", strcat(s3, s1));
21 }

```

s1 = Happy
s2 = New Year
strcat(s1, s2) = Happy New Year
strncat(s3, s1, 6) = Happy
strcat(s3, s1) = Happy Happy New Year

String Comparison Functions (<string.h>)

- **Purpose:** Compare the contents of two strings.
- **Library:** String-Handling Library (<string.h>)
- **Key Functions:**
 - strcmp -> Compares two entire strings.
 - strncmp -> Compares up to n characters of two strings.

Functions strcmp and strncmp

- strcmp

- **Prototype:** `int strcmp(const char *s1, const char *s2);`
- **Purpose:** Performs a lexicographical (character-by-character) comparison of two strings.
- **Return Value:**
 - 0 if the strings are **equal**.
 - **Negative Value** if s1 is **less than** s2.
 - **Positive Value** if s1 is **greater than** s2.

- strncmp

- **Prototype:** `int strncmp(const char *s1, const char *s2, size_t n);`
- **Purpose:** Compares up to the first n characters of two strings.
- **Key Behavior:** Stops comparing if a null character is encountered in either string before n characters are compared.

How String Comparison Works (Lexicographical)

- **Concept:** Strings are compared based on the **numeric character codes** (e.g., ASCII, Unicode) of their characters.
- **Process:**
 - Compare the first character of each string.
 - If they are equal, move to the next character.
 - Repeat until a difference is found or the end of a string is reached.
- **The "Alphabet" is an Ordered List:**
 - 'A' (65) < 'B' (66) < ... < 'Z' (90)
 - 'a' (97) < 'b' (98) < ... < 'z' (122)
 - **Crucially:** Uppercase letters have **lower** numeric values than lowercase letters ('A' (65) < 'a' (97)).
- **Example:**
 - "Apple" vs. "Banana": 'A' < 'B', so "Apple" is **less than** "Banana".
 - "apple" vs. "Apple": 'a' > 'A', so "apple" is **greater than** "Apple".

Understanding the Return Value

- The exact negative/positive value is **implementation-dependent**.
- **Common Behaviors:**
 - Return **-1** if $s1 < s2$, and **1** if $s1 > s2$.
- **What matters for programming:** Use the return value in conditional statements by checking if it is zero, less than zero, or greater than zero.

```
if (strcmp(string1, string2) == 0) {  
    printf("The strings are equal.\n");  
}  
else if (strcmp(string1, string2) < 0) {  
    printf("string1 comes before string2.\n");  
}  
else {  
    printf("string1 comes after string2.\n");  
}
```

```

1 // fig08_13.c
2 // Using functions strcmp and strncmp
3 #include <stdio.h>
4 #include <string.h>
5
6 int main(void) {
7     const char *s1 = "Happy New Year"; // initialize char pointer
8     const char *s2 = "Happy New Year"; // initialize char pointer
9     const char *s3 = "Happy Holidays"; // initialize char pointer
10
11     printf("s1 = %s\ns2 = %s\ns3 = %s\n\n%s%2d\n%s%2d\n%s%2d\n\n",
12           s1, s2, s3,
13           "strcmp(s1, s2) = ", strcmp(s1, s2),
14           "strcmp(s1, s3) = ", strcmp(s1, s3),
15           "strcmp(s3, s1) = ", strcmp(s3, s1));
16
17     printf("%s%2d\n%s%2d\n%s%2d\n",
18           "strncmp(s1, s3, 6) = ", strncmp(s1, s3, 6),
19           "strncmp(s1, s3, 7) = ", strncmp(s1, s3, 7),
20           "strncmp(s3, s1, 7) = ", strncmp(s3, s1, 7));
21 }

```

```

s1 = Happy New Year
s2 = Happy New Year
s3 = Happy Holidays

```

```

strcmp(s1, s2) = 0
strcmp(s1, s3) = 1
strcmp(s3, s1) = -1

```

```

strncmp(s1, s3, 6) = 0
strncmp(s1, s3, 7) = 1
strncmp(s3, s1, 7) = -1

```


String Search Functions (<string.h>) - Overview

- **Purpose:** The string-handling library provides functions to search for characters and substrings within a string.
- **Library:** String-Handling Library (<string.h>)
- **Key Functions:**
 - strchr - Find first occurrence of a character.
 - strrchr - Find last occurrence of a character.
 - strstr - Find a substring.
 - strpbrk - Find first occurrence of any character from a set.
 - strtok - Break a string into tokens.

Character Search Functions

- `strchr`

- **Prototype:** `char *strchr(const char *str, int c);`
- **Purpose:** Finds the **first occurrence** of character `c` in string `str`.
- **Returns:** Pointer to the character, or `NULL` if not found.

- `strrchr`

- **Prototype:** `char *strrchr(const char *str, int c);`
- **Purpose:** Finds the **last occurrence** of character `c` in string `str`.
- **Returns:** Pointer to the character, or `NULL` if not found.

```
char s[] = "A zoo has zebras";
```

```
char *ptr1 = strchr(s, 'z'); // Points to first 'z' in "zoo"
```

```
char *ptr2 = strrchr(s, 'z'); // Points to 'z' in "zebras"
```

```

1 // fig08_14.c
2 // Using function strchr
3 #include <stdio.h>
4 #include <string.h>
5
6 int main(void) {
7     const char *string = "This is a test"; // initialize char pointer
8     char character1 = 'a';
9     char character2 = 'z';
10
11     // if character1 was found in string
12     if (strchr(string, character1) != NULL) { // can remove "!= NULL"
13         printf("\'%c\' was found in \"%s\".\n", character1, string);
14     }
15     else { // if character1 was not found
16         printf("\'%c\' was not found in \"%s\".\n", character1, string);
17     }
18
19     // if character2 was found in string
20     if (strchr(string, character2) != NULL) { // can remove "!= NULL"
21         printf("\'%c\' was found in \"%s\".\n", character2, string);
22     }
23     else { // if character2 was not found
24         printf("\'%c\' was not found in \"%s\".\n", character2, string);
25     }
26 }

```

'a' was found in "This is a test".
 'z' was not found in "This is a test".


```

1 // fig08_17.c
2 // Using function strrchr
3 #include <stdio.h>
4 #include <string.h>
5
6 int main(void) {
7     const char *s1 = "A zoo has many animals including zebras";
8     int c = 'z'; // character to search for
9
10    printf("%s '%c' %s\n\n"%s\"",
11           "Remainder of s1 beginning with the last occurrence of character",
12           c, "is:", strrchr(s1, c));
13 }

```

Remainder of s1 beginning with the last occurrence of character 'z' is:
"zebras"

Substring and Character-Set Search Functions

- `strstr`

- **Prototype:** `char *strstr(const char *str1, const char *str2);`
- **Purpose:** Finds the **first occurrence** of the entire substring `str2` in `str1`.
- **Returns:** Pointer to the beginning of the substring, or `NULL` if not found.

- `strpbrk`

- **Prototype:** `char *strpbrk(const char *str1, const char *str2);`
- **Purpose:** Finds the **first occurrence** of **any character** from `str2` in `str1`.
- **Returns:** Pointer to the character, or `NULL` if not found.

```
char s[] = "abcdef";
```

```
char *p1 = strstr(s, "cde"); // Points to "cdef"
```

```
char *p2 = strpbrk(s, "xd"); // Points to 'd' in "def"
```

```

1 // fig08_19.c
2 // Using function strstr
3 #include <stdio.h>
4 #include <string.h>
5
6 int main(void) {
7     const char *string1 = "abcdefabcdef";
8     const char *string2 = "def"; // string to search for
9
10    printf("string1 = %s\nstring2 = %s\n\n%s\n%s%s\n", string1, string2,
11           "The remainder of string1 beginning with the",
12           "first occurrence of string2 is: ", strstr(string1, string2));
13 }

```

```

string1 = abcdefabcdef
string2 = def

```

```

The remainder of string1 beginning with the
first occurrence of string2 is: defabcdef

```

// Extract words separated by delimiters

```
char text[] = "name=value;category=user;role=admin";
```

```
char *delimiters = "=";
```

```
char *start = text;
```

```
while (*start) {  
    char *end = strpbrk(start, delimiters);  
    if (end) {  
        printf("Token: %.*s\n", (int)(end - start), start);  
        start = end + 1;  
    } else {  
        printf("Last token: %s\n", start);  
        break;  
    }  
}
```

```
Token: name  
Token: value  
Token: category  
Token: user  
Token: role  
Last token: admin
```

```
char *password = "MyPass123@";  
char *forbidden = "!@#$%^&*"; // Characters not allowed  
char *found = strpbrk(password, forbidden);  
if (found) {  
    printf("Invalid character '%c' found in password\n", *found);  
}
```

```
Invalid character '@' found in password
```


Tokenizing Function: strtok

- **Prototype:** `char *strtok(char *str, const char *delimiters);`
- **Purpose:** Breaks a string into a series of tokens using specified delimiters.
- **Critical Warning:** **Modifies the original input string** by replacing delimiters with `'\0'`. Always work on a copy if you need the original string later

How to Use strtok

- **First Call**

- Provide the string to tokenize and the delimiter string.
- strtok finds the first token, null-terminates it, and saves an internal pointer.

- **Subsequent Calls**

- Call strtok with NULL as the first argument to continue tokenizing the same string.
- The function returns NULL when no more tokens are found.

```
char string[] = "This is a test";
```

```
char *token = strtok(string, " "); // Gets "This"
```

```
while (token != NULL) {
```

```
    printf("%s\n", token);
```

```
    token = strtok(NULL, " "); // Gets "is", "a", "test"
```

```
}
```

```

1 // fig08_20.c
2 // Using function strtok
3 #include <stdio.h>
4 #include <string.h>
5
6 int main(void) {
7     char string[] = "This is a sentence with 7 tokens";
8
9     printf("The string to be tokenized is:\n%s\n\n", string);
10    puts("The tokens are:");
11
12    char *tokenPtr = strtok(string, " "); // begin tokenizing sentence
13
14    // continue tokenizing sentence until tokenPtr becomes NULL
15    while (tokenPtr != NULL) {
16        printf("%s\n", tokenPtr);
17        tokenPtr = strtok(NULL, " "); // get next token
18    }
19 }

```

```

The string to be tokenized is:
This is a sentence with 7 tokens

The tokens are:
This
is
a
sentence
with
7
tokens

```


Function: strlen

- **Prototype:** `size_t strlen(const char *str);`
- **Purpose:** Returns the number of characters in a string, **excluding** the terminating null character ('\0').
- **Key Point:** The length is the count of characters *before* the null terminator. This is crucial for loops and memory allocation

strlen vs. sizeof for Strings

- **strlen:** A function that calculates the length of the string *content* at runtime.
- **sizeof:** An operator that returns the total size of the *array* in bytes at compile-time.

- **Critical Difference:**

```
char str[100] = "Hello";
```

```
printf("strlen: %zu\n", strlen(str)); // Output: 5 (string length)
```

```
printf("sizeof: %zu\n", sizeof(str)); // Output: 100 (array size)
```

- **Remember:** Always use `strlen` to get the length of the string itself, and `sizeof` to get the total capacity of the character array.